

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина Методы трансляции

ОТЧЕТ
к лабораторной работе № 5
на тему

ИНТЕРПРЕТАЦИЯ ИСХОДНОГО КОДА

Студент

П. Н. Носкович

Преподаватель

Н. Ю. Гриценко

Минск 2025

СОДЕРЖАНИЕ

1 Постановка задачи.....	3
2 Краткие теоретические сведения.....	4
3 Результаты выполнения лабораторной работы.....	5
Заключение	6
Список использованных источников	7
Приложение А(обязательное) Листинг программного кода	8

1 ПОСТАНОВКА ЗАДАЧИ

Цель данной лабораторной работы заключается в разработке интерпретатора, который будет выполнять программы, используя абстрактное синтаксическое дерево, полученное на предыдущем этапе. В процессе интерпретации особое внимание уделяется выполнению операций, управлению состоянием переменных и контролю за корректностью значений. Интерпретатор будет обрабатывать семантические ошибки, такие как несоответствие типов и использование неинициализированных переменных, а также предоставлять информативные сообщения об ошибках. Это поможет упростить отладку исходного кода и улучшить взаимодействие пользователя с интерпретируемым языком.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Интерпретация программы представляет собой процесс непосредственного выполнения исходного кода программы, в отличие от компиляции, которая преобразует код в машинный код для последующего выполнения. Интерпретатор читает и выполняет код построчно или небольшими блоками, не создавая отдельный исполняемый файл.

Ключевым аспектом интерпретации является её динамичность. Интерпретатор анализирует и выполняет код во время выполнения программы, что позволяет оперативно выявлять ошибки и вносить изменения в код без необходимости перекомпиляции всей программы. Это особенно полезно при разработке и отладке программ, когда требуется быстрая обратная связь [1].

Процесс интерпретации включает в себя несколько этапов. Сначала интерпретатор анализирует исходный код, проверяя его синтаксис и семантику. Если обнаруживаются ошибки, выполнение программы останавливается и выдается сообщение об ошибке. Если код синтаксически и семантически корректен, интерпретатор приступает к его выполнению. На этом этапе происходит выделение памяти для переменных, выполнение операций и вызов функций [2].

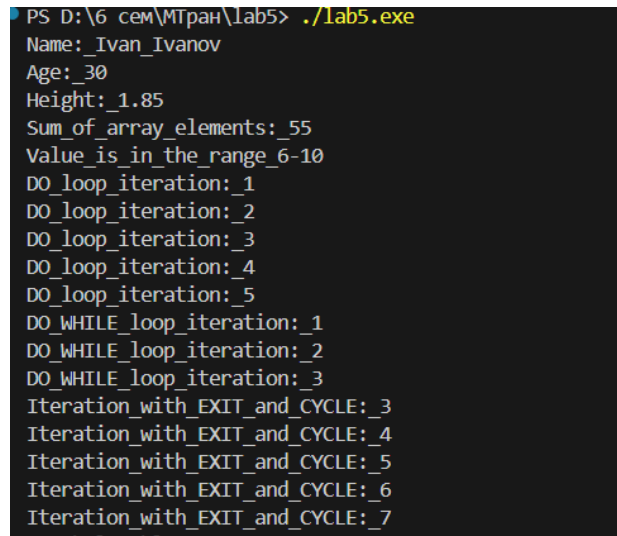
Одним из преимуществ интерпретации является её кроссплатформенность. Интерпретируемые языки, такие как *Python* и *JavaScript*, могут выполняться на различных операционных системах, если на них установлен соответствующий интерпретатор. Это упрощает разработку и распространение программ, так как не требуется создавать отдельные версии для каждой платформы.

Однако у интерпретации есть и недостатки. Интерпретируемые программы обычно выполняются медленнее, чем компилируемые, так как интерпретатору приходится анализировать и выполнять код во время выполнения. Кроме того, интерпретируемые программы требуют наличия интерпретатора на целевой системе, что может быть проблемой в некоторых случаях.

Несмотря на эти недостатки, интерпретация широко используется в различных областях, таких как веб-разработка, скриптовые языки и научные вычисления. Её динамичность и кроссплатформенность делают её привлекательным выбором для многих задач.

3 РЕЗУЛЬТАТЫ ВЫПОЛНЕНИЯ ЛАБОРАТОРНОЙ РАБОТЫ

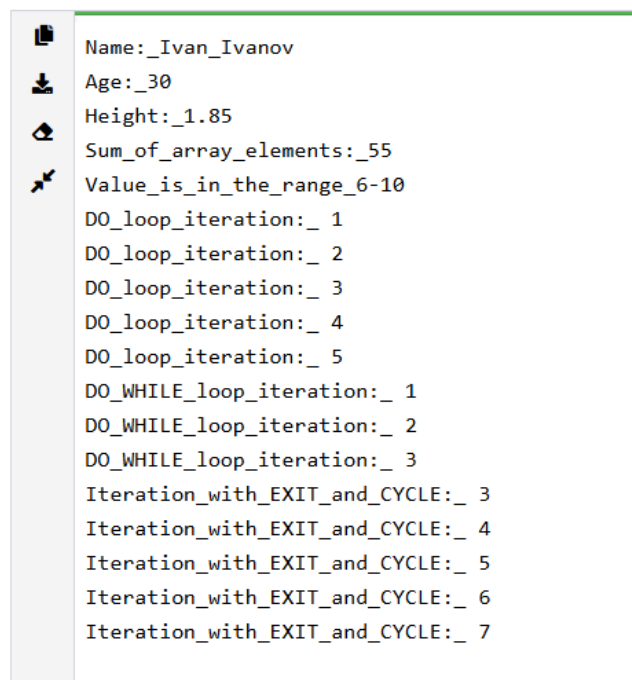
Во время выполнения лабораторной работы была выполнена интерпретация языка *RНР*. Скриншот результатов выполнения программы представлен на рисунке 3.1.



```
PS D:\6 сем\МТран\lab5> ./lab5.exe
Name: Ivan_Ivanov
Age: 30
Height: 1.85
Sum_of_array_elements: 55
Value_is_in_the_range_6-10
DO_loop_iteration: 1
DO_loop_iteration: 2
DO_loop_iteration: 3
DO_loop_iteration: 4
DO_loop_iteration: 5
DO_WHILE_loop_iteration: 1
DO_WHILE_loop_iteration: 2
DO_WHILE_loop_iteration: 3
Iteration_with_EXIT_and_CYCLE: 3
Iteration_with_EXIT_and_CYCLE: 4
Iteration_with_EXIT_and_CYCLE: 5
Iteration_with_EXIT_and_CYCLE: 6
Iteration_with_EXIT_and_CYCLE: 7
```

Рисунок 3.1 – Итог работы интерпретатора

Результат выполнения самой программы на языке *RНР* представлен на рисунке 3.2.



```
Name: Ivan_Ivanov
Age: 30
Height: 1.85
Sum_of_array_elements: 55
Value_is_in_the_range_6-10
DO_loop_iteration: 1
DO_loop_iteration: 2
DO_loop_iteration: 3
DO_loop_iteration: 4
DO_loop_iteration: 5
DO_WHILE_loop_iteration: 1
DO_WHILE_loop_iteration: 2
DO_WHILE_loop_iteration: 3
Iteration_with_EXIT_and_CYCLE: 3
Iteration_with_EXIT_and_CYCLE: 4
Iteration_with_EXIT_and_CYCLE: 5
Iteration_with_EXIT_and_CYCLE: 6
Iteration_with_EXIT_and_CYCLE: 7
```

Рисунок 3.2 – Результат выполнения программы на языке *RНР*

ЗАКЛЮЧЕНИЕ

В процессе выполнения лабораторной работы по созданию интерпретатора были изучены ключевые аспекты интерпретации программного кода, включая построение выполнения, динамический анализ и обработку ошибок. Были разработаны алгоритмы для разбора исходного кода, проверки синтаксической и семантической корректности, а также для выполнения операций и вызова функций. Особое внимание было уделено обработке ошибок, чтобы предоставлять пользователю понятные и информативные сообщения о проблемах в коде, что способствует более эффективной отладке и улучшению качества программного обеспечения.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Интерпретация программы [Электронный ресурс]. – Режим доступа: <https://edu.vsu.ru/mod/resource/view.php?id=25354> – Дата доступа: 18.03.2025

[2] Введение в теорию интерпретаторов [Электронный ресурс]. – Режим доступа: <https://habr.com/ru/articles/515420/> – Дата доступа: 18.03.2025

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг программного кода

```
MODULE SymbolTable
  IMPLICIT NONE
  INTEGER, PARAMETER :: max_symbols = 1000
  INTEGER, PARAMETER :: max_functions = 50
  INTEGER, PARAMETER :: max_scope_depth = 10
  INTEGER, PARAMETER :: max_errors = 100

  TYPE Symbol
    CHARACTER(LEN=32) :: name
    CHARACTER(LEN=10) :: type
    CHARACTER(LEN=10) :: scope
    INTEGER :: scope_level
    CHARACTER(LEN=256) :: value
  END TYPE Symbol

  TYPE Function
    CHARACTER(LEN=32) :: name
    CHARACTER(LEN=10) :: return_type
    INTEGER :: arg_count
    CHARACTER(LEN=10), DIMENSION(10) :: arg_types
  END TYPE Function

  TYPE Class
    CHARACTER(LEN=32) :: name
  END TYPE Class

  INTEGER, PARAMETER :: max_classes = 50
  TYPE(Class), DIMENSION(max_classes) :: classes
  INTEGER :: class_count = 0

  TYPE(Symbol), DIMENSION(max_symbols) :: symbols
  TYPE(Function), DIMENSION(max_functions) :: functions
  CHARACTER(LEN=256), DIMENSION(max_errors) :: errors
  INTEGER :: symbol_count = 0
  INTEGER :: function_count = 0
  INTEGER :: current_scope_level = 0
  INTEGER :: error_count = 0
  INTEGER :: current_function_index = 0
  LOGICAL :: expected_braces = .FALSE.
  LOGICAL :: inside_function = .FALSE.
  LOGICAL :: inside_class = .FALSE.

CONTAINS

SUBROUTINE ProcessCodeBlock(lines, num_lines)
  CHARACTER(LEN=256), DIMENSION(:), INTENT(IN) :: lines
  INTEGER, INTENT(IN) :: num_lines
  INTEGER :: current_line

  current_line = 1
  DO WHILE (current_line <= num_lines)
    CALL ProcessCodeLine(lines(current_line), lines, num_lines,
current_line)
    current_line = current_line + 1
  END DO
END SUBROUTINE ProcessCodeBlock
```



```

SUBROUTINE ProcessClassDeclaration(line)
  CHARACTER(LEN=*), INTENT(IN) :: line
END SUBROUTINE ProcessClassDeclaration

SUBROUTINE ProcessFunctionDeclaration(line)
  CHARACTER(LEN=*), INTENT(IN) :: line
END SUBROUTINE ProcessFunctionDeclaration

END MODULE SymbolTable


PROGRAM Interpreter
  USE SymbolTable
  IMPLICIT NONE
  CHARACTER(LEN=256) :: line
  CHARACTER(LEN=256), DIMENSION(1000) :: lines
  INTEGER :: num_lines = 0, ios

  OPEN(UNIT=10, FILE='INPUT.TXT', STATUS='OLD', IOSTAT=ios)
  IF (ios /= 0) THEN
    PRINT *, "Error opening file"
    STOP
  END IF

  DO
    READ(10, '(A)', IOSTAT=ios) line
    IF (ios /= 0) EXIT
    num_lines = num_lines + 1
    lines(num_lines) = line
  END DO
  CLOSE(10)

  CALL ProcessCodeBlock(lines, num_lines)
  CALL PrintSymbolTable()
  CALL PrintErrors()

END PROGRAM Interpreter

```